

PREFACE FOR FACULTY

Infinite Impulse Response (IIR) filters contain feedback loops. Because past output states are recycled, the arrangement of arithmetic operations directly affects memory requirements (number of delay registers) and numerical stability under quantization.

Pedagogical Strategy:

1. Express the general IIR transfer function as a cascade of all-zero and all-pole blocks:
$$H(z) = H_{\text{zeros}}(z) \cdot H_{\text{poles}}(z).$$
 2. Derive **Direct Form I**: Implements zeros first, then poles $\implies M + N$ delays (non-canonical).
 3. Derive **Direct Form II (Canonical Form)**: Reverses the order (poles first, then zeros), allowing both subsystems to share a single internal state delay line $w[n] \implies \max(M, N)$ delays.
 4. Derive **Transposed Direct Form II**: Reverses signal flow graph branches to eliminate multi-input accumulator bottlenecks and enable pipelining.
-

1. LEARNING OBJECTIVES

By the end of this lecture, students will be able to:

1. **Formulate** difference equations and signal flow graphs for Direct Form I, Direct Form II, and Transposed Direct Form II.
 2. **Calculate** minimum delay register counts for canonical vs. non-canonical structures.
 3. **Analyze** intermediate node scaling to prevent internal register overflow.
 4. **Compare** Direct Form structures across critical path latency and hardware complexity.
-

2. MATHEMATICAL FOUNDATIONS

2.1 General IIR Transfer Function

$$H(z) = \frac{\sum_{k=0}^M b_k z^{-k}}{1 + \sum_{k=1}^N a_k z^{-k}} = \frac{Y(z)}{X(z)}$$

Difference Equation:

$$y[n] = \sum_{k=0}^M b_k x[n-k] - \sum_{k=1}^N a_k y[n-k]$$

2.2 Direct Form I Realization

- Implements feedforward path (M delays) and feedback path (N delays) separately.

- **Total Delays:** $M + N$.
- **Total Multipliers:** $M + N + 1$.
- **Total Adders:** $M + N$.

2.3 Direct Form II (Canonical Realization)

Let $W(z) = \frac{X(z)}{1 + \sum_{k=1}^N a_k z^{-k}}$, then $Y(z) = \left(\sum_{k=0}^M b_k z^{-k} \right) W(z)$. In the time domain:

$$w[n] = x[n] - \sum_{k=1}^N a_k w[n - k]$$

$$y[n] = \sum_{k=0}^M b_k w[n - k]$$

- **Total Delays:** $\max(M, N)$ (Canonical).
- **Total Multipliers:** $M + N + 1$.
- **Total Adders:** $M + N$.

2.4 Transposed Direct Form II

Applying flow graph reversal to Direct Form II:

$$\begin{aligned} y[n] &= b_0 x[n] + v_1[n - 1] \\ v_k[n] &= b_k x[n] - a_k y[n] + v_{k+1}[n - 1], \quad k = 1, 2, \dots, N - 1 \\ v_N[n] &= b_N x[n] - a_N y[n] \end{aligned}$$

3. WORKED NUMERICAL EXAMPLES

Example 17.1: Direct Form I and II Realization

Problem: Realize the second-order IIR filter:

$$H(z) = \frac{2 + 3z^{-1} + 4z^{-2}}{1 - 0.6z^{-1} + 0.25z^{-2}}$$

in (a) Direct Form I, (b) Direct Form II.

Solution: Here $b_0 = 2, b_1 = 3, b_2 = 4$ and $a_1 = -0.6, a_2 = 0.25$. **(a) Direct Form I:**

- Requires $M + N = 2 + 2 = 4$ delay elements.
- Difference Equation: $y[n] = 2x[n] + 3x[n - 1] + 4x[n - 2] + 0.6y[n - 1] - 0.25y[n - 2]$.

(b) Direct Form II (Canonical):

- Requires $\max(2, 2) = 2$ delay elements $w[n - 1]$ and $w[n - 2]$.
- State Equation: $w[n] = x[n] + 0.6w[n - 1] - 0.25w[n - 2]$.
- Output Equation: $y[n] = 2w[n] + 3w[n - 1] + 4w[n - 2]$.

4. UNIVERSITY EXAMINATION QUESTIONS & MARKING RUBRIC

Question 1 (15 Marks)

(a) Realize the system function in Direct Form I and Direct Form II:

$$H(z) = \frac{1 + 2z^{-1} + z^{-2}}{1 - 0.75z^{-1} + 0.125z^{-2}}$$

(8 Marks) **(b)** Derive the Transposed Direct Form II structure for the same transfer function. Why is the transposed structure preferred for high-speed VLSI implementation? *(7 Marks)*

Model Answer & Step-by-Step Marking Rubric:

- **Part (a):**
 - Direct Form I SFG (4 delays, 5 multipliers, 4 adders) *(4 Marks)*
 - Direct Form II Canonical SFG (2 shared delays, state equations) *(4 Marks)*
- **Part (b):**
 - Transposed state equations and neatly drawn Transposed DF-II SFG *(4 Marks)*
 - Explanation: Multipliers share common input $x[n]$ and $y[n]$, adder tree is distributed between delays, eliminating long critical-path propagation delays *(3 Marks)*

5. PYTHON VERIFICATION SCRIPT

```
import scipy.signal as signal

b = [1, 2, 1]
a = [1, -0.75, 0.125]
impulse = [1, 0, 0, 0, 0, 0]
y = signal.lfilter(b, a, impulse)
print("Impulse Response y[n]:", y)
```